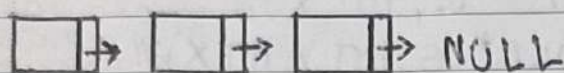


21/10/25

* listas encadeadas

é uma forma de armazenar um conjunto de elementos, alternativa a vetores.

É composta por nós que apontam para o próximo elemento:



As características dos vetores:

- Um elemento de uma lista encadeada não é indexado;
- os elementos de uma lista podem não ser armazenados de forma contígua na memória; e
- pode ser mais adequada para inserção e remoção de elementos, ou seja, quando o conjunto de dados varia.

TAD (Tipo Abstrato de Dado)

O TAD é um modelo de dado cujo comportamento é definido por sua estrutura, valores e operações.

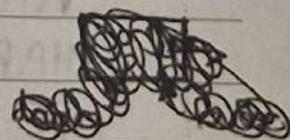
Sempre usaremos um TAD para modelar e implementar as estruturas de dados que veremos daqui por diante.

~~Ex - Lista encadeada com ponteiros~~

TYPEDEF INT ESTRELINHA;

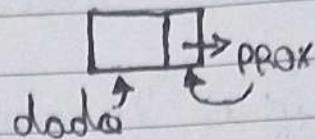
modelos
do dado

↳ nome do novo
tipo de dado



TAD - Listas Encadeadas com nó cabeça

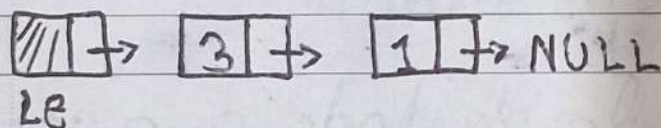
```
typedef struct CELULA {  
    int DADO;  
    struct CELULA *PROX;  
} CELULA;
```



```
CELULA *CRIA_LISTA();  
void INSERE_INICIO(CELULA *LE, int x); // insere x no início da lista  
void INSERE_DEPOIS(CELULA *LE, int y, int x); // insere x depois de y  
int REMOVE_INICIO(CELULA *LE); // se não tiver na lista, não faz nada  
int REMOVE_ELEMENTO(CELULA *LE, int x); // remove x da lista se não tiver  
CELULA *BUSCA(CELULA *LE, int x); // na lista, não altera  
int LISTA_VAZIA(CELULA *LE);  
void DESTROI_LISTA(CELULA *LE);
```

Obs.:

- ① CELULA *LE é um ponteiro para o início da lista.
- ② Usaremos listas encadeadas com nó cabeça



- ③ Lista vazia:

Registro

```
struct PESSOA {  
    char nome[100];  
    char CPF[12];  
    int idade;  
};
```

opcional
atributos

STRUCT PESSOA P;

P.IDADE = 30;

↳ : *acessa um campo da struct*

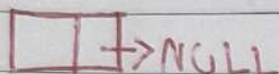
CELULA *CRIA.LISTA(){

CELULA *LE = MALLOC(sizeof(CELULA));

LE → PROX = NULL; // → = (*LE).PROX

RETURN LE;

}

 → NULL

na main

CELULA *LE

CELULA *LE = CRIA.LISTA();

Por que não implementar assim?

CELULA CRIA.LISTA(){

CELULA LE; → *alocação estática ou automática*

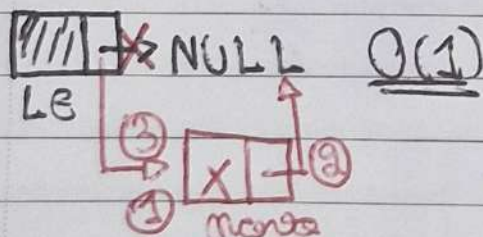
LE.PROX = NULL;

RETURN LE; → *retorna uma cópia*

}

- *Faz o dobro do trabalho*
- *Não permite as chamadas por referência que serão feitas adiante.*

insersão



~~VOID INSERE_INICIO(CELULA *LE, INT X){~~
VOID INSERE_INICIO(CELULA *LE, INT X){
CELULA *NOVO = MALLOC(sizeof(CELULA));
NOVO → DADO = X;
NOVO → PROX = LE → PROX;
LE → PROX = NOVO;
}